

Problem: World-Class Mobile Checkout

Context

Checkout is where intent becomes revenue. A Gametime fan buying tickets on the native app is often doing it minutes before the game starts, standing outside the venue, on patchy cell signal. Every extra tap, every form field, every moment of “did that work?” costs conversion at the most expensive possible point in the funnel — after we’ve already paid to acquire the fan and they’ve already decided to buy.

The single biggest lever in modern checkout is express payment. A fan with Apple Pay shouldn’t type a card number. A fan on Android with Google Pay shouldn’t either. A fan buying \$400 of playoff tickets might want to split the payment with Affirm. But these options aren’t universal — each one has rules about where it can appear, and showing a payment method that can’t actually work is worse than not showing it at all. On a native app, the fan can also background or get pulled out of your app mid-payment (a biometric prompt, an Affirm redirect, a phone call) — and coming back has to just work.

The Problem

You’re building Gametime’s checkout screen from scratch in React Native. The fan has already picked their seats; an order summary is on screen. Now they need to pay — and the set of ways they can pay depends on the platform they’re on, what’s provisioned on their device, and what they’re buying:

- Apple Pay is only available on iOS, and only if the device actually has a card provisioned in Wallet
- Google Pay is only available on Android, and only if the device has Google Pay set up
- Affirm is only available when the purchase total is over \$100
- Credit card is always available, and is the fallback for everyone else

Express methods need to live up to the name: tapping Apple Pay, Google Pay, or Affirm should complete the purchase in that flow — no bouncing the fan back to a submit button. The card form is the opposite: it should refuse to let a fan submit anything that isn’t a valid card, and it should make getting to valid as painless as possible on a native keyboard.

Your Challenge

Build a checkout screen that:

- Shows the right payment methods to the right fan — Apple Pay, Google Pay, and Affirm appear only when their eligibility rules are met. The rules above are the spec; how you detect them is up to you.
- Handles credit cards properly — Card number, expiry, and CVC with real validation (formatting, card-type detection, Luhn, expiry-in-the-future), using native inputs with sensible keyboardType/textContentType behavior. The checkout button must not allow submission until the card is valid.
- Makes express checkout actually express — Selecting Apple Pay, Google Pay, or Affirm completes the purchase in one interaction. No second submit tap. This includes surviving the app being backgrounded mid-flow — a biometric prompt, an Affirm redirect through a webview/browser, or the OS interrupting the app — and resuming cleanly when the fan comes back.
- Talks to a payment API you design — Mock the backend yourself. The payment SDKs can be stubbed; the API contract between your checkout and your mock backend is part of what we're evaluating. It should handle at least one failure path (a declined card, a cancelled Apple Pay sheet) and the UI should recover gracefully — including not double-charging if the app is killed and relaunched mid-request.

You define the scope beyond this. We care about how you reason about eligibility detection, payment state across app-lifecycle interruptions, and form UX — more than visual polish.

Some questions worth thinking through:

- What happens to your checkout state when the OS backgrounds your app (an AppState change) during a redirect or biometric prompt — and what happens if the fan force-quits and relaunches instead of just backgrounding?
- What states can the checkout exist in (idle, validating, processing, declined, succeeded), and what does the fan see in each — including “we don't know yet, we're checking”?
- When does card validation fire — on keystroke, on blur, on submit — and how does that interact with autofill from the device keychain or Google autofill?
- If the order total changes (quantity, fees), how does Affirm's eligibility react?

Constraints and Notes

- Use React Native (Expo or bare, your choice) targeting iOS and Android.
- Stub the payment SDKs — do not integrate real Apple Pay, Google Pay, or Affirm sandbox credentials. A stub that mimics the real interaction shape (a native sheet, a capability check, an authorization token) is what we want.
- Build the mock payment API yourself. In-process mocks, a local server, or a mock native module are all fine — but there should be a real request/response boundary with a contract you can defend.
- Include a way to simulate environments for review — a dev menu, hidden settings screen, or feature flag that forces “iOS,” “Android,” “no provisioned cards,” etc., so we can review your eligibility logic without four devices. Your real detection logic should still exist and be the default.
- Use obviously fake card numbers (e.g., 4242 4242 4242 4242) for testing. Never real ones.
- Simulator/emulator testing is fine — you don’t need physical devices or a paid Apple developer account.

What We’re Looking For

- A working checkout where payment method visibility matches the eligibility rules
- A credit card form with real validation and a submit button that enforces it
- Express flows that complete the purchase without a second submit, and survive an app-lifecycle interruption mid-flow
- A mock API with a defensible contract, including at least one failure path handled in the UI
- Clean, idiomatic React Native code in a repo we can review

Time Expectation

We expect this to take roughly 3 hours using modern development tools. Prioritize eligibility logic, payment state handling across app-lifecycle events, and form validation over visual design.

Submission

Create a GitHub repo with your solution. Include a README that explains:

- What you built and how to run it (Expo Go, simulator, emulator — note which platforms you tested)

- How eligibility detection works, and how to use the environment simulator
- Your mock API contract and why you shaped it that way
- How payment state flows from “tap” to “confirmed” for express vs. card, and what happens across an app backgrounding/kill-and-relaunch mid-flow
- What tradeoffs you made
- What you’d do differently with more time